SENG 31242 - SYSTEM DESIGN PROJECT

BK Restaurant Management System

An integrated delivery, POS & management platform for three branches

Group 01

BK Restaurant & BK Bar

University of Kelaniya

Team Members:

SE/2022/020- Sachin Lakshitha
SE/2022/023- Duvini Nimethra
SE/2022/026- Dinuja Ranaweera
SE/2022/030- Nethmi Rajapaksha

Client:
Mr. N. C. Sanjeewa De Silva
BK Restaurant & BK Bar

INTRODUCTION

Project Overview

BK Restaurant Management System is a web- and mobile-based platform that digitises and centralises operations for BK Restaurant and BK Bar across three branches in Ambalangoda and Kahawa, Sri Lanka.

Current Issues

- Paper-based order management
- WhatsApp communication between branches
- Manual, end-of-day billing process
- Difficult, error-prone inventory tracking

Our Solution

A single, centralised digital management system that replaces WhatsApp, paper ledgers, and disconnected branch operations with one integrated platform spanning POS, kitchen display, delivery tracking, inventory, employees, and reporting.

THE PROBLEM

Three Branches, Running on WhatsApp and Paper

BK Restaurant & BK Bar operate across Ambalangoda and Kahawa with no shared system between branches.

1. Missed & Duplicated Orders	**Orders taken over WhatsApp are manually tracked, causing confusion and lost revenue during busy hours.**
2. Zero Delivery Visibility	**Riders are dispatched with no real-time tracking — delays go unnoticed until the customer calls.**
3. Inventory Stockouts	**Stock levels are updated manually, so low inventory is discovered too late, disrupting service.**
4. Manual Financial Reconciliation	**Cross-branch reports are stitched together by hand in spreadsheets — slow and error-prone.**
5. Paper-Based Staffing	**Shifts and schedules run on paper or informal chats, leading to conflicts and gaps in coverage.**

SCOPE

In Scope vs Out of Scope

In Scope

- Web-based POS for counter orders & billing
- Kitchen Display System (KDS) in real time
- Flutter mobile app for delivery riders + live GPS
- Automated inventory with low-stock alerts
- Digital supplier stock-request workflow
- Employee attendance, leave & shift scheduling
- Centralised owner dashboard across branches
- Role-based access control for 5 user types

Out Scope

- In-house payment gateway development (3rd-party integration only)
- Customer-facing ordering app (orders still placed via phone/WhatsApp per UC-06)
- iOS support for the Rider App (Android-only for this phase)
- Hardware procurement (POS terminals, printers — assumed available)
- Automated route optimisation for riders (manual/GPS-guided only)
- Franchise or multi-tenant support beyond the three current branches

GOALS

System Objectives

Derived from client interviews and the formal Functional Requirements (FR-01 to FR-18) in the SRS.

Centralised Order Management

One system for dine-in, takeaway & delivery orders

Real-Time Kitchen Communication

Live Kitchen Display System (KDS) replaces shouted/paper orders

Delivery Tracking

Live GPS visibility of riders and ETAs

Inventory Management

Auto stock deduction & low-stock alerts

Employee Management

Shifts, attendance and leave in one place

Multi-Branch Monitoring

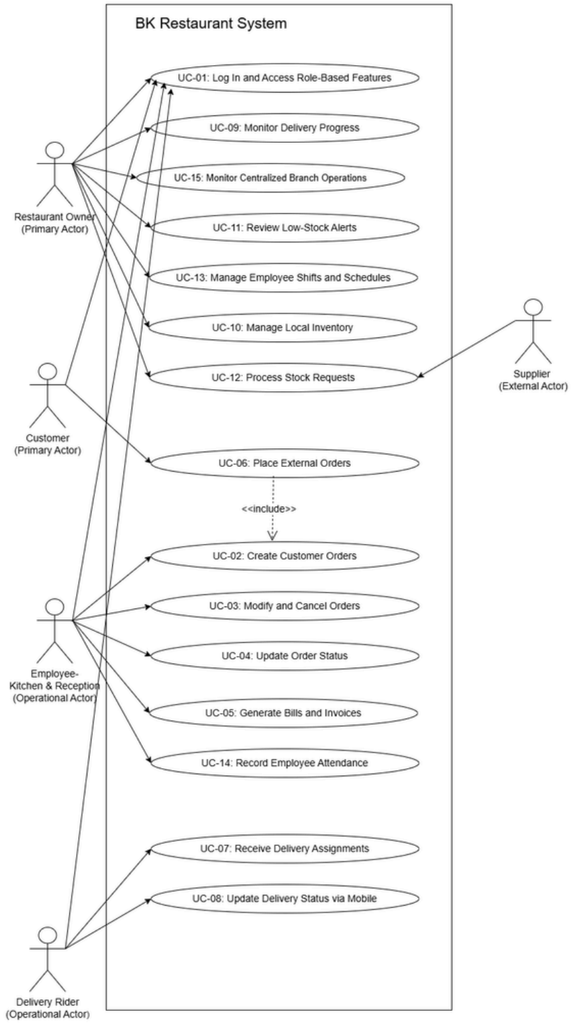
Owner visibility across all branches

Digital Billing & Reporting

Automated invoices and sales analytics

Use Case Diagram — System Boundary

15 use cases across 5 actors define the complete scope of system interaction.



Actor Details

- **Owner**

  Monitors deliveries, branch ops, inventory & stock requests, employee shifts — UC-09, 10–13, 15

- **Customer**

  Places external orders via phone/WhatsApp — UC-06 (includes UC-02)

- **Reception & Kitchen**

  Creates, modifies & updates orders, generates bills, records attendance — UC-02–05, 14

- **Delivery Rider**

  Receives assignments and updates delivery status on mobile — UC-07, 08

- **Supplier (external)**

  Processes inbound stock requests — UC-12

ACTORS

System Users

Restaurant Owner

Monitors all 3 branches, views analytics, approves decisions

Branch Manager

Manages local inventory, staff shifts and stock requests

Reception Staff

Creates orders, generates bills at the POS counter

Kitchen Staff

Views & updates order status on the Kitchen Display System

Delivery Rider

Receives assignments, navigates, updates delivery status

Role-Based Access Control — each user signs in to a role-specific dashboard and only sees the features authorised for that role (FR-17).

ARCHITECTURE

High-Level Architecture

Pattern Selected: Layered (N-Tier) Client-Server Architecture, combined with a modular, feature-based backend.

Presentation Layer

React.js web app (POS/KDS/Owner) + Flutter mobile app (Rider)

API Layer

Node.js + Express REST API — single entry point for all clients

Business Logic Layer

Validation, status transitions, bill totals, stock thresholds

Data Access Layer

ORM mapping domain entities to PostgreSQL

Database Layer

Shared PostgreSQL database + Redis cache layer

Core Technologies

- React.js
- Flutter
- Node.js
- PostgreSQL
- Redis

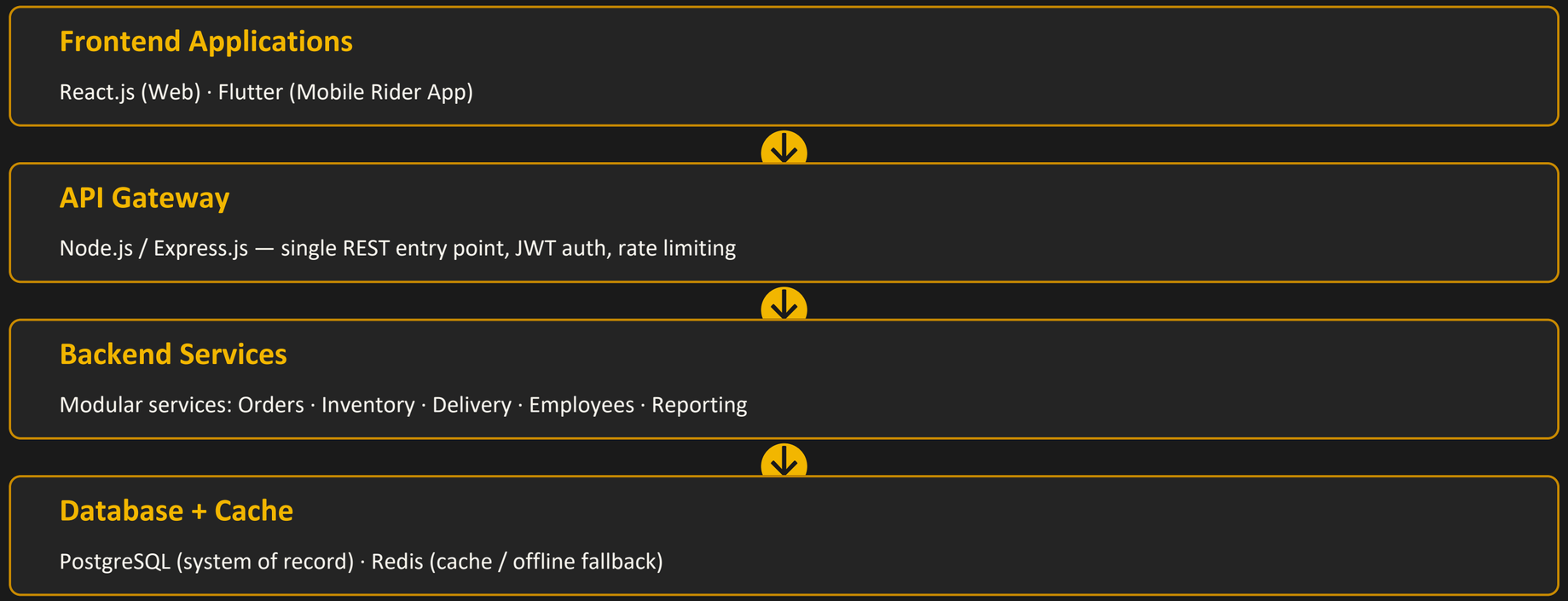
Why This Pattern

- **Maintainability** - independent modules per feature
- **Scalability** - stateless API scales horizontally
- **Performance** - Redis caching meets the 2s target

ARCHITECTURE

Component Diagram

Frontend Applications → API Gateway → Backend Services → Database & Cache



Authentication | Orders | Inventory | Delivery | Employees | Reporting

IMPLEMENTATION

Technology Stack

Web Frontend

React.js + Tailwind CSS

POS, KDS & Owner Dashboard UI

Mobile App

Flutter

Cross-platform rider app (Android)

Backend

Node.js + Express

Non-blocking I/O for real-time updates

Database

PostgreSQL

Transactional data, referential integrity

Cache

Redis

Menu & dashboard caching, offline fallback

Authentication

JWT

Stateless, role-based access (FR-17/18)

Maps

Google Maps Platform

Live GPS tracking & navigation

Hosting

AWS / Railway.app

Cost-effective, scalable cloud hosting

DATA MODEL

Database Design

Database: Single shared PostgreSQL database across all branches, partitioned by a branchId foreign key where applicable (SDS §4.4).

Main Entities

User	**Role**	**Branch**
Customer	**Order**	**OrderItem**
MenuItem	**Invoice**	**Delivery**
DeliveryTracking	**InventoryItem**	**StockRequest**
Supplier	**Employee / Attendance**	**Shift**
LeaveRequest		

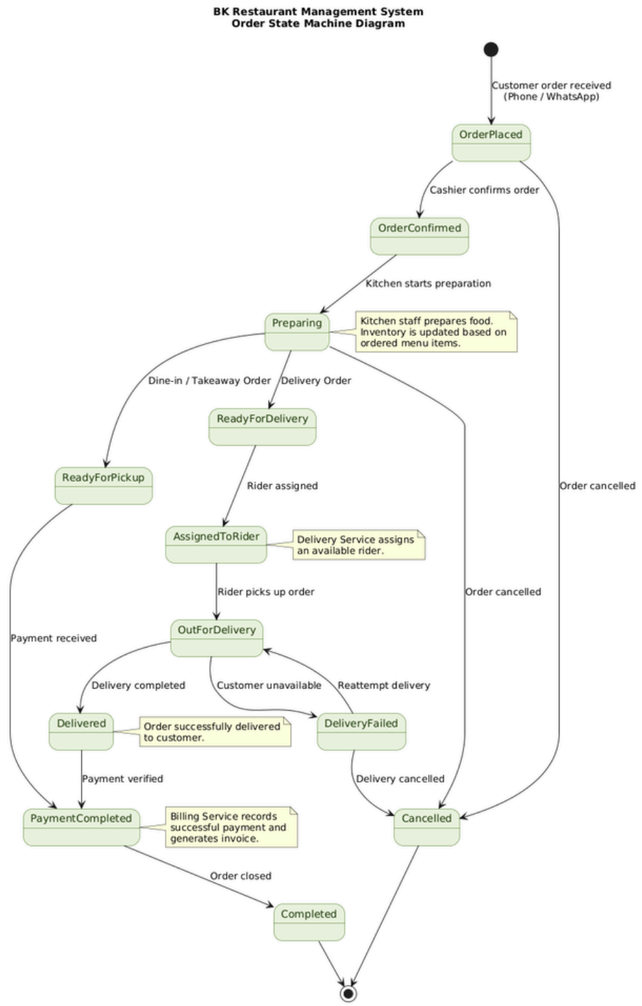
Key Relationships

Branch
1—* User, InventoryItem
Order
1—* OrderItem, 1 Invoice
Order
0..1 Delivery
Delivery
1—* DeliveryTracking
InventoryItem
— StockRequest
Supplier
1—* StockRequest

DETAILED DESIGN

Order Lifecycle — State Machine

Every order follows an enforced state path — invalid jumps (e.g. Pending ➔ Completed) are rejected by the API.



KEY TRANSITIONS

- Placed ➔ Confirmed

  Cashier confirms the order; inventory check runs.

- Preparing ➔ Ready

  Splits by type: dine-in/takeaway ➔ pickup, delivery ➔ rider assignment.

- AssignedToRider ➔ OutForDelivery

  Rider picks up the order; GPS tracking begins.

- Delivered ➔ PaymentCompleted

  Payment verified; invoice generated by the Billing Service.

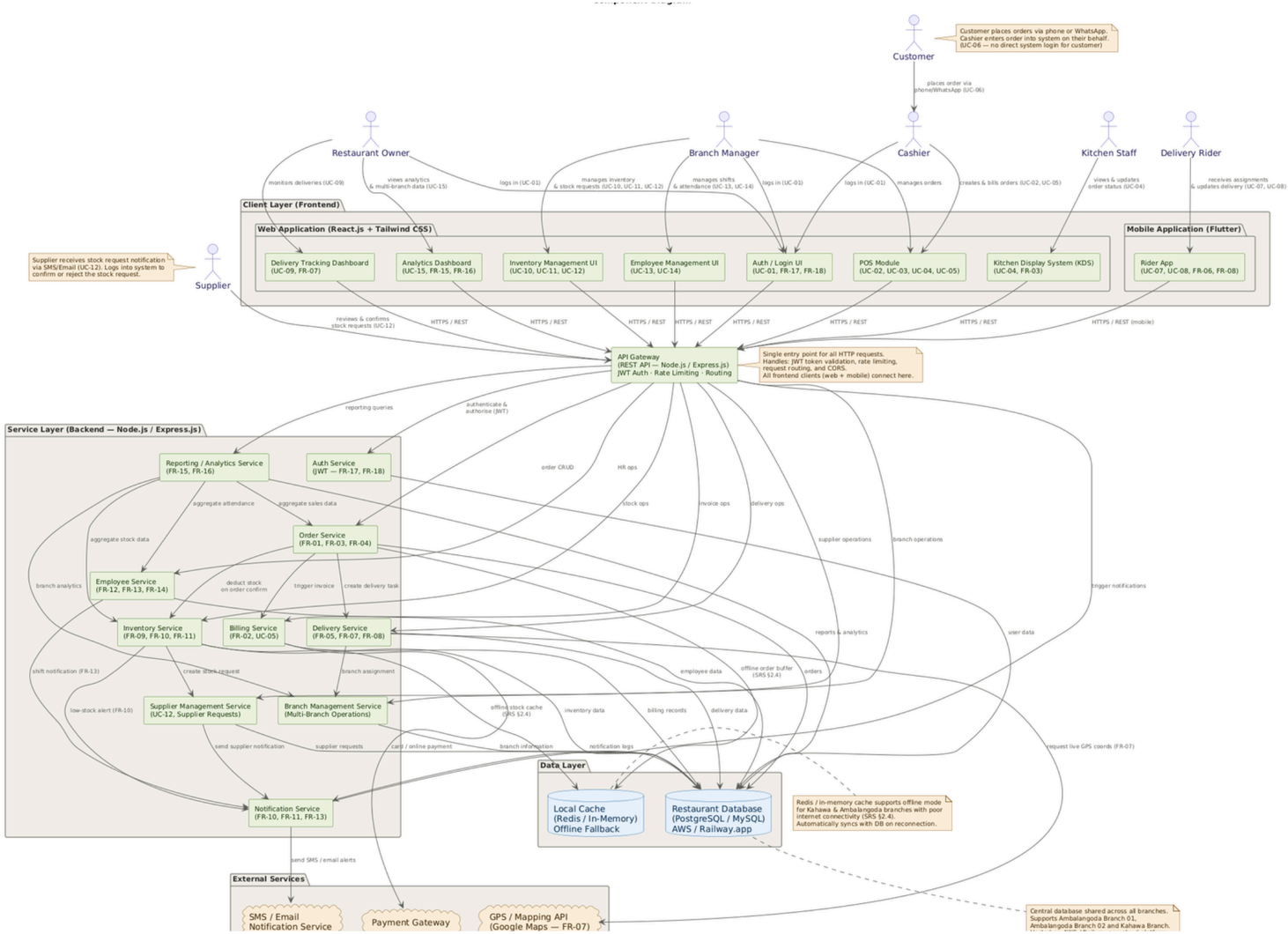
- Any state ➔ Cancelled

  Cancellation permitted until dispatch, per business rules in UC-03.

ARCHITECTURAL DESIGN

Component Diagram — Full System

Client layer, API gateway, modular backend services, data layer, and external integrations.



DETAILED DESIGN

Core Data Model

A single shared PostgreSQL database, partitioned by branchId where access needs to stay branch-scoped.

User

id, name, role, branchId, passwordHash

Order

id, branchId, type, status, total, items[]

Invoice

id, orderId, total, tax, timestamp

Delivery

id, orderId, riderId, status, gpsLocation

InventoryItem

id, branchId, name, quantity, threshold

StockRequest

id, itemId, supplierId, status, quantity

Shift

id, employeeId, branchId, start, end

Input Validation

id, employeeId, date, checkIn, checkOut

UI/UX

User Interface Design

Major Interfaces — wireframed in full in the SDS

POS Dashboard

Order entry, billing, payment for reception staff

Kitchen Display System

Live order queue with status updates for kitchen staff

Rider Mobile App

Assignment, navigation & delivery status for riders

Inventory Dashboard

Stock levels, low-stock alerts, supplier requests

Owner Dashboard

Multi-branch analytics, sales & delivery monitoring

Simple UI

Easy Navigation

Real-Time Updates

INTEGRATION

External Interfaces

Third-Party Integrations (SDS 6.1)

Google Maps Platform

Live GPS rider tracking, navigation, delivery-zone validation

Thermal Receipt Printer

Physical bill printing at the POS counter

Email / SMS Notifications

Digital invoices, supplier alerts, shift notifications

Cloud Hosting Services

AWS / Railway.app — managed API, DB & cache hosting

Benefits: Live Tracking · Digital Invoices · Automated Notifications

INTERFACE DESIGN

Internal Module Interfaces

A versioned REST API (/api/v1/...) using JSON payloads and JWT-based authentication headers.

Authentication	• **POST /auth/login | POST /auth/refresh | GET /auth/me**	*Consumers: Web Client, Mobile Client*
Orders	• **POST /orders | PATCH /orders/{id}/status | POST /orders/{id}/finalize**	*Consumers: POS UI, KDS UI*
Inventory	• **GET /inventory | GET /inventory/alerts | POST /stock-requests Module**	*Consumers: Inventory UI, Orders*
Delivery	• **POST /deliveries | PATCH /deliveries/{id}/status | GET /deliveries/active** *Dashboard*	*Consumers: Rider App, Owner*
Employee	• **POST /shifts | POST /attendance | POST /leave-requests**	*Consumers: Scheduling UI, Attendance UI*
Reporting	• **GET /analytics/sales | GET /analytics/inventory | GET /analytics/delivery**	*Consumers: Owner Dashboard*

SECURITY DESIGN

OWASP TOP 10 - How the Design Responds

A01 — Broken Access Control
Centralised RBAC middleware on every request

A02 — Cryptographic Failures
Bcrypt hashing + HTTPS/TLS everywhere

A03 — Injection
Parameterised queries / ORM, schema validation

A04 — Insecure Design
Explicit state machines prevent invalid states

A05 — Security Misconfiguration
Env-based secrets; debug disabled in prod

A06 — Vulnerable Components
Dependencies tracked via npm audit / Dependabot

A07 — Auth Failures
Expiring JWTs + refresh tokens; rate-limited login

A08 — Data Integrity Failures
Audit logging on all critical record changes

A09 — Logging & Monitoring Failures
Audit + application logs on auth & key events

A10 — Server-Side Request Forgery
Fixed config-defined endpoints, no user-supplied URLs

SECURITY

Security Design

Security Features — mapped to OWASP Top 10:2021 (SDS §7)

JWT Authentication

Stateless, signed tokens with expiry & refresh

Role-Based Access Control

Middleware enforces role + branch scope on every endpoint

Password Encryption (bcrypt)

Salted hashes — passwords never stored in plain text

HTTPS Communication

All client-API traffic encrypted in transit (TLS)

Audit Logging

User ID, action & timestamp recorded for key changes

Input Validation

Schema validation + parameterised queries prevent injection

QUALITY ATTRIBUTES

Non-Functional Requirements

8 categories defined in SRS (NFR-01 to NFR-22)

Performance

- Fast response time
- Real-time processing

Security

- Secure authentication
- Role-based access control

Usability

- Allow multiple users simultaneously
- Clear, easy navigation

Reliability

- Remain available with minimal downtime.
- Data consistency across branches

REQUIREMENTS

Core Functional Requirements

8 key functional areas, drawn from FR-01–FR-18 in the SRS.

Order Management

FR-01

Billing & Invoice Generation

FR-02

Kitchen Display System

FR-03

Delivery Assignment

FR-05

GPS Tracking

FR-07

Inventory Management

FR-09

Employee Management

FR-12–14

Reporting Dashboard

FR-15/16

CONCLUSION

Expected Benefits & What's Next

Expected Benefits

- Centralised management across all branches
- Faster order processing, fewer missed orders
- Better inventory control with automated alerts
- Improved, GPS-tracked delivery visibility
- Better decision-making through live analytics

Future Enhancements

Online Customer Ordering

Direct customer-facing ordering channel

AI-Based Sales Prediction

Forecast demand to reduce stockouts & wastage

Advanced Analytics Dashboard

Deeper cross-branch business intelligence

References

1. Software Requirements Specification (SRS) — BK Restaurant Management System, Group 01
2. Software Design Specification (SDS) — BK Restaurant Management System, Group 01
3. IEEE Std 1016-1998 — IEEE Recommended Practice for Software Design Descriptions
4. ISO/IEC/IEEE 42010:2011 — Systems and Software Engineering: Architecture Description
5. OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks (owasp.org/Top10)
6. React.js Documentation (react.dev)
7. Flutter Documentation (flutter.dev)
8. PostgreSQL Documentation (postgresql.org/docs)

THANK YOU!

Q & A